

ScholarRAG

Academic Retrieval-Augmented Generation System

Technical Documentation

Generated: 2026-09-30

1. Abstract

ScholarRAG is a state-of-the-art Retrieval-Augmented Generation (RAG) system specifically designed for academic and professional knowledge retrieval. Traditional generative AI models often suffer from hallucination and lack the ability to cite their sources. ScholarRAG solves this by providing a unified, local-first ingestion pipeline that supports various document formats (PDFs, PPTXs, TXT) and processes them into semantically rich vectors.

By leveraging cutting-edge embedding models and a reciprocal rank fusion mechanism (combining dense vector search with sparse BM25 retrieval), ScholarRAG ensures that the most relevant context is retrieved for any given query. A cross-encoder reranker further refines these results to maximize precision. Finally, the retrieved context is passed to a Large Language Model (LLM) to generate an accurate, highly-detailed response that

includes page-level citations, allowing users to verify the generated claims against the original source documents.

2. Tech Stack

ScholarRAG utilizes a modern, decoupled client-server architecture.

2.1. Frontend

- **Core Framework:** React (with TypeScript) and Vite.
- **Styling & UI:** TailwindCSS, Radix UI, and Lucide React icons. It features dynamic theming, a glassmorphism design, and a responsive layout.
- **State Management:** Zustand (for workspace and state persistence).
- **Communication:** Native `fetch` API utilizing Server-Sent Events (SSE) for real-time streaming of LLM tokens and metadata.

2.2. Backend

- **Core Framework:** FastAPI (Python), run via Uvicorn.
- **Database:** SQLite (via SQLAlchemy and SQLAlchemyModel) for storing workspace configurations, document metadata, and chat history.
- **Document Parsing:** PyMuPDF (`fitz`) for PDFs/Images and `python-pptx` for presentations, unified through a custom preprocessing pipeline.

2.3. AI & RAG Pipeline

- **Embedding Model:** BAAI/bge-small-en-v1.5 (loaded locally via HuggingFace `sentence-transformers`).
- **Vector Store:** ChromaDB for high-performance dense semantic retrieval.
- **Sparse Retrieval:** BM25Okapi (via `rank_bm25`) for exact-keyword and lexical matching.
- **Reranking Engine:** BAAI/bge-reranker-base (Cross-Encoder, loaded locally) for high-precision result ordering.
- **Generative LLM:** Extensible `LLMProvider` interface. Currently defaults to **Gemini** (`gemini-3.5-flash`), with built-in support for **OpenAI** (`gpt-4o-mini`) and **Ollama** (`llama3.2:3b`).

3. Architecture & Flow

The system is logically divided into two primary pipelines: **Ingestion** and **Generation**.

3.1. Document Ingestion Pipeline

When a user uploads a document to a Workspace, the following steps occur: 1. **Format Unification**: Documents (PPTX, DOCX, TXT) are converted to a standardized format (PDF) for uniform processing. 2. **Text Extraction**: PyMuPDF extracts the raw text along with spatial heuristics to determine headings and hierarchical structure. 3. **Chunking**: The extracted text is passed through a Recursive Character Text Splitter. Chunks are sized optimally (e.g., 500-1000 tokens) with overlap to preserve context boundaries. 4. **Vectorization**: Each chunk is passed through the BAAI/bge-small-en-v1.5 embedding model to generate a dense vector representation. 5. **Storage**: The dense vectors are indexed into ChromaDB, while the raw text is added to the BM25 index. Document metadata (page numbers, chunk IDs) is stored in SQLite.

3.2. Query & Generation Pipeline

When a user asks a question, the backend executes a highly optimized retrieval flow:

1. **Query Embedding:** The user's question is embedded using the same `bge-small` model.
2. **Hybrid Retrieval:**
 - **Dense Search:** ChromaDB retrieves the top-K chunks based on cosine similarity.
 - **Sparse Search:** BM25 retrieves chunks based on keyword overlap.
3. **Reciprocal Rank Fusion (RRF):** The results from both retrieval methods are mathematically fused to balance semantic meaning with keyword presence.
4. **Cross-Encoder Reranking:** The top results from RRF are paired with the original question and passed through the `bge-reranker-base` model. This model outputs a precise relevance score (0 to 1), and the chunks are re-ordered.
5. **Prompt Construction:** The top reranked chunks are formatted into a rigid system prompt, explicitly instructing the LLM to only use the provided context and to output citations in a specific `[doc_id]` format.
6. **LLM Generation:** The prompt is sent to the configured LLM Provider.
7. **Streaming Response:** The LLM's response is streamed back to the React frontend via SSE. The frontend parses the stream to instantly render text, citations, and metadata without waiting for the full generation to complete.

4. Directory Structure

The repository is structured into isolated frontend and backend services.

```
ScholarRAG/
├── backend/                                # FastAPI Application
│   ├── app/
│   │   ├── api/                            # API Route Handlers (FastAPI Routers)
│   │   ├── core/                          # Configuration, Logging, Error Handling
│   │   ├── database/                      # SQLite DB schemas and ChromaDB initialization
│   │   ├── ingestion/                    # Document Parsing, Chunking, and Format Conversion
│   │   ├── generation/                   # LLM Providers (OpenAI, Ollama, Gemini)
│   │   ├── services/                     # Business logic tying DB and models together
│   │   └── main.py                        # Application Entrypoint
│   ├── .env                              # Environment Variables & API Keys
│   └── requirements.txt                   # Python Dependencies
```

```
■
■■■ frontend/                # React UI Application
■   ■■■ src/
■   ■   ■■■ components/      # Reusable React UI Components (e.g., Workspace, Navbar)
■   ■   ■■■ lib/             # Utility functions, Zustand stores, API constants
■   ■   ■■■ App.tsx          # Main React Component
■   ■   ■■■ main.tsx         # React DOM Entrypoint
■   ■■■ index.html           # HTML Template
■   ■■■ package.json         # Node Dependencies
■   ■■■ tailwind.config.js   # TailwindCSS Configuration
■   ■■■ vite.config.ts       # Vite Bundler Configuration
```

5. Testing & Performance

5.1. Component Testing

- **Backend Ingestion Test (`test_upload.py`):** A standalone script that bypasses the API to directly test the ingestion layer. It tests format conversion, text extraction, and vector indexing.
- **Backend Generation Test (`test_qa.py`):** A script that tests the end-to-end RAG pipeline, ensuring that the retrieval mechanisms (RRF, Reranking) correctly locate the relevant chunks, and that the LLM produces a valid SSE stream with citations.

5.2. Performance Metrics

ScholarRAG has been tuned for high performance while minimizing the local hardware footprint:

- * **Memory Optimization:** The system relies on quantization and efficient models. The `bge-small` embedding model requires < 500 MB of VRAM/RAM. The `bge-reranker-base` model requires ~1.1 GB.
- * **Latency:**
 - **Retrieval Phase:** Dense + Sparse search completes in < 50ms.
 - **Reranking Phase:** Cross-encoder scoring of top 20 documents completes in ~200-400ms on a modern CPU.
 - **Time-to-First-Token (TTFT):** When using Google Gemini (`gemini-3.5-flash`), TTFT is typically under 1 second after the user submits the query.
- * **Throughput:** The FastAPI backend utilizes asynchronous I/O (`asyncio`, `httpx`, and `StreamingResponse`) allowing the server to handle dozens of concurrent user streams without blocking the event loop. Heavy CPU tasks (like embedding and reranking) are dispatched to separate thread pools using `asyncio.to_thread()`.

5.3. Scalability

The architecture is designed to scale horizontally. Since the `chromaDB` instance is separated from the application state, it can be seamlessly swapped for a distributed vector database (like Milvus or Pinecone). Similarly, the local SQLite database can be upgraded to PostgreSQL for production deployments requiring multi-node support.